

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Системи штучного інтелекту»

Робота №1

Тема «Розпізнавання образів на основі метричної класифікації»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Калюжна О.С.

2025

МЕТА

Вивчити та засвоїти на практиці метричні методи розпізнавання образів у просторі ознак, навчитися створювати програмні засоби, що реалізують методи метричної класифікації, порівняти результати роботи метричної класифікації з іншими видами класифікації.

ЗАВДАННЯ

Частина 1.

1. Ознайомитися з конспектом лекцій та рекомендованою літературою. На мові програмування Python написати програму, що реалізує процедури для навчання та емуляції за методом еталонів.

2. Згідно з номером студента за журналом для відповідного номера варіанта V сформувати навчаючу вибірку $\langle x, y \rangle$ обсягом S екземплярів x^s , $s=1, 2, \dots, S$, що характеризуються N ознаками x_j^s , $j=1, 2, \dots, N$, та зіставити кожному екземпляру значення цільової ознаки y^s :

$$x_j^s = \begin{cases} jV - 0,1s, & j = 1, 5, 9, \dots, \\ 0,01jV^{-1} + 0,3s, & j = 2, 4, 6, \dots, \\ j\text{rand}, & j = 3, 7, 11, \dots; \end{cases} \quad y^s = \begin{cases} 0, & (x_1^s)^2 + (x_2^s)^2 < V^2 + 0,04S^2, \\ 1, & (x_1^s)^2 + (x_2^s)^2 \geq V^2 + 0,04S^2; \end{cases}$$
$$S = \begin{cases} 10V, & V < 10, \\ 5V, & 10 \leq V < 20, \\ 3V, & V \geq 20; \end{cases} \quad N = \begin{cases} 5V, & V < 7, \\ 4V, & 7 \leq V < 10, \\ 3V, & 10 \leq V < 20, \\ 2V, & V \geq 20; \end{cases}$$

де rand - випадкове число в діапазоні $[0, 1]$.

3. Виконати нормування навчаючої вибірки даних.

4. На основі пронормованої вибірки побудувати розпізнаючу модель за методом еталонів, тобто визначити координати центрів класів (еталонів) у просторі ознак C_j^q , де q - номер класу, j - номер ознаки. Зафіксувати час навчання.

5. На основі побудованої моделі для екземплярів навчаючої вибірки виконати розпізнавання, тобто визначити розрахункові номери класів ys^* . Зафіксувати час розпізнавання.

6. Обчислити помилку розпізнавання, визначити ймовірність прийняття правильного рішення та ймовірність прийняття помилкового рішення для побудованої моделі.

7. У попередньо сформованій вибірці залишити тільки одну ознаку, номер якої у попередній вибірці дорівнює V , а також у центрів еталонів класів залишити тільки V -ту координату C_qV .

8. Для нової вибірки та нових еталонів виконати розпізнавання, тобто визначити розрахункові номери класів ys^* . Зафіксувати час розпізнавання.

9. Обчислити помилку розпізнавання, визначити ймовірність прийняття правильного рішення та ймовірність прийняття помилкового рішення для нової моделі.

10. Порівняти результати проведених експериментів, зробити висновки щодо впливу параметрів навчаючої вибірки на характеристики процесів навчання та розпізнавання.

Частина 2.

1. Описати вибірку, надати короткий опис наявним в вибірці ознакам, проаналізувати їх, визначити малозначущі і видалити їх (за наявності).

2. Дослідити наявність пропущених даних, їх кількість. За потреби усунути нестачу будь-яким прийнятним методом. Обґрунтувати вибір.

3. Виконати розпізнавання наступними алгоритмами з пакету «Sklearn» і порівняти їх з результатами метричної класифікації (методом найближчих сусідів):

- методом найближчих сусідів (KNeighborsClassifier);
- випадковий ліс (RandomForestClassifier);
- метод логістичної регресії (LogisticRegression);
- метод опорних векторів (SVC).

4. Результати розпізнавання відобразити у вигляді графіку, проаналізувати їх, зробити припущення як їх можна покращити.

ТЕОРІЯ

Пакет `scikit-learn` призначений для формування моделей розпізнавання образів та їх використання.

Вибірку даних потрібно розбити на навчальну та тестову функцією `train_test_split`.

Непотрібні ознаки можна прибрати функцією `drop()`.

Метод еталонів реалізується моделлю `KMeans`. Вона приймає кількість кластерів та шукає їх центри. Тоді відносить кожен екземпляр до певного кластеру.

Навчання моделі проводиться функцією `fit()`. Розпізнавання виконується функцією `predict()`.

Нормування вибірки можна виконати функцією `normalize()`.

Засікти час виконання можна функцією `time.time()` або `time.perf_counter()`.

Отримати помилку розпізнавання можна функцією `score()`.

ВИКОНАННЯ

1 Метод еталонів

```
import os
import random
import time
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, normalize

current_folder = os.path.dirname(os.path.abspath(__file__))
file_path = os.path.join(current_folder, "train.csv")
data = pd.read_csv(file_path)

# 1.1 Еталони

needed = "Name, Age, Survived".split(",")
data = data[needed].dropna()
print(data)

survived_label = data.Survived.to_numpy()
data = data.drop(["Survived", "Name"], axis=1)

X_train, X_test, y_train, y_test = train_test_split(data,
survived_label)
model = KMeans(n_clusters=3)
model.fit(X_train)
print(f"{model.cluster_centers_ =}")
prediction_results = model.predict(X_test)
print(f"{prediction_results =}")

# 1.2 Вибірка

def get_target_feature(row: list[int]):
    if row[0] ** 2 + row[1] ** 2 < variant**2 + 0.04 * dataset_size**2:
        return 0
    elif row[0] ** 2 + row[1] ** 2 >= variant**2 + 0.04 *
dataset_size**2:
        return 1

variant = 19
dataset_size = 5 * variant
features_count = 3 * variant

ones_range = range(1, dataset_size, 4)
twos_range = range(2, dataset_size, 4)
threes_range = range(3, dataset_size, 4)
features_holder = []
targets_holder = []
for sample_number in range(1, dataset_size):
    sample_features = []
```

```

        for feature_number in range(1, features_count):
            if feature_number in ones_range:
                prediction_results = feature_number * variant - 0.1 *
sample_number
            elif feature_number in twos_range:
                prediction_results = 0.01 * feature_number * variant**-1
            elif feature_number in threes_range:
                prediction_results = feature_number * random.random()
            sample_features.append(prediction_results)

        targets_holder.append(get_target_feature(row=sample_features))
        features_holder.append(sample_features)
features = np.array(features_holder)
target = np.array(targets_holder)
print(f"{features = }")
print(f"{target = }")

df = pd.DataFrame(data=features)
df["target"] = target
print(df)

# 1.3 Нормування

normalized_features = normalize(features)
print(f"{normalized_features = }")

# 1.4 Побудова моделі

features_train, features_test, targets_train, targets_test =
train_test_split(
    normalized_features, target
)

start_time = time.perf_counter()
model = KMeans(n_clusters=features_count)
model.fit(features_train)
end_time = time.perf_counter()
learning_time = end_time - start_time
print(f"{learning_time = }")
print(f"{model.cluster_centers_ = }")

# 1.5 Розпізнавання

start_time = time.perf_counter()
prediction_results = model.predict(features_test)
print(f"{prediction_results = }")
end_time = time.perf_counter()
prediction_time = end_time - start_time
print(f"{prediction_time = }")

# 1.6 Помилка

original_prediction_error = model.score(features_test)
print(f"{original_prediction_error = }")
original_error_probability = original_prediction_error / dataset_size *
features_count
original_correct_probability = 1 - original_error_probability
print(f"{original_error_probability = }")

```

```

print(f"{original_correct_probability = }")

# 1.7 Нова вибірка

single_features = [row[variant] for row in features_holder]
print(f"{single_features = }")
single_center = int(prediction_results[variant])
print(f"{single_center = }")
single_target = get_target_feature(row=single_features)
print(f"{single_target = }")

# 1.8 Розпізнавання

single_features = np.array(single_features).reshape(-1, 1)
model = model.fit(single_features)
prediction_results = model.predict(single_features)
print(f"{prediction_results = }")

# 1.9 Помилка

prediction_error = model.score(single_features)
print(f"{prediction_error = }")
short_error_probability = prediction_error / len(single_features) * 1
short_correct_probability = 1 - short_error_probability
print(f"{short_error_probability = }")
print(f"{short_correct_probability = }")

# 1.10 Порівняння

prediction_error_difference = abs(original_prediction_error -
prediction_error)
error_probability_difference = abs(original_error_probability -
short_error_probability)
correct_probability_difference = abs(
    original_correct_probability - short_correct_probability
)
print(f"{prediction_error_difference = }")
print(f"{error_probability_difference = }")
print(f"{correct_probability_difference = }")

# 1.10.1 Порівняння таблицею

errors_table = pd.DataFrame(
    {
        "Error": [original_error_probability, short_error_probability],
        "Correct": [original_correct_probability,
short_correct_probability],
    },
    index=["Original", "Shortened"],
)
print(errors_table)

```

```

      Name  Age  Survived
0      Braund, Mr. Owen Harris  22.0      0
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  38.0      1
2      Heikkinen, Miss. Laina  26.0      1
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)  35.0      1
4      Allen, Mr. William Henry  35.0      0
..      ...      ...      ...
885      Rice, Mrs. William (Margaret Norton)  39.0      0
886      Montvila, Rev. Juozas  27.0      0
887      Graham, Miss. Margaret Edith  19.0      1
889      Behr, Mr. Karl Howell  26.0      1
890      Dooley, Mr. Patrick  32.0      0

[714 rows x 3 columns]
model.cluster_centers_ = array([[ 7.31986486],
 [49.29927007],
 [26.58024691]])
prediction_results = array([1, 2, 1, 1, 2, 2, 2, 1, 2, 1, 1, 1, 1, 2, 2, 2, 1, 1, 2, 2, 2, 2,
 0, 1, 2, 0, 1, 2, 2, 2, 2, 1, 2, 1, 2, 2, 2, 2, 2, 0, 2, 2, 2, 0,
 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 0, 1, 1, 1, 1, 1, 2, 1, 0, 2, 2,
 2, 2, 2, 0, 2, 2, 2, 2, 2, 0, 2, 2, 2, 2, 2, 1, 1, 1, 2, 2, 2, 1,
 0, 1, 1, 0, 1, 1, 2, 0, 2, 1, 2, 1, 0, 2, 0, 0, 2, 1, 2, 2, 2, 2,
 2, 0, 2, 0, 2, 2, 2, 0, 2, 1, 0, 2, 1, 1, 2, 1, 2, 2, 0, 2, 0, 2,
 1, 1, 2, 1, 2, 0, 2, 1, 2, 2, 2, 2, 1, 1, 1, 2, 2, 1, 2, 0, 2, 2,
 2, 2, 0, 2, 2, 2, 0, 2, 2, 1, 1, 2, 2, 1, 0, 2, 2, 2, 1, 1, 1,
 2, 2, 1], dtype=int32)
features = array([[1.89000000e+01, 1.05263158e-03, 2.16905410e+00, ...,
 2.84210526e-02, 4.68592552e+01, 4.68592552e+01],
 [1.88000000e+01, 1.05263158e-03, 2.96273604e+00, ...,
 2.84210526e-02, 8.67097688e+00, 8.67097688e+00],
 [1.87000000e+01, 1.05263158e-03, 2.42213433e+00, ...,
 2.84210526e-02, 4.49994102e+01, 4.49994102e+01],
 ...,
 [9.80000000e+00, 1.05263158e-03, 2.66901519e+00, ...,
 2.84210526e-02, 5.07640984e+01, 5.07640984e+01],
 [9.70000000e+00, 1.05263158e-03, 2.83759544e+00, ...,
 2.84210526e-02, 4.63815866e+01, 4.63815866e+01],
 [9.60000000e+00, 1.05263158e-03, 4.71184533e-01, ...,
 2.84210526e-02, 4.88965494e+01, 4.88965494e+01]], shape=(94, 56))
target = array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0, 0, 0, 0])
      0      1      2      3      4      ...      52      53      54      55  target
0  18.9  0.001053  2.169054  2.169054  94.9  ...  1006.9  0.028421  46.859255  46.859255  0
1  18.8  0.001053  2.962736  2.962736  94.8  ...  1006.8  0.028421  8.670977  8.670977  0
2  18.7  0.001053  2.422134  2.422134  94.7  ...  1006.7  0.028421  44.999410  44.999410  0
3  18.6  0.001053  0.028826  0.028826  94.6  ...  1006.6  0.028421  8.162770  8.162770  0
4  18.5  0.001053  1.845657  1.845657  94.5  ...  1006.5  0.028421  40.253012  40.253012  0
..      ...      ...      ...      ...      ...      ...      ...      ...      ...      ...
89  10.0  0.001053  2.015173  2.015173  86.0  ...  998.0  0.028421  24.244090  24.244090  0
90  9.9  0.001053  2.539240  2.539240  85.9  ...  997.9  0.028421  46.795751  46.795751  0
91  9.8  0.001053  2.669015  2.669015  85.8  ...  997.8  0.028421  50.764098  50.764098  0
92  9.7  0.001053  2.837595  2.837595  85.7  ...  997.7  0.028421  46.381587  46.381587  0
93  9.6  0.001053  0.471185  0.471185  85.6  ...  997.6  0.028421  48.896549  48.896549  0

[94 rows x 57 columns]

```

Рисунок 1.1 — Виконання


```

normalized_features = array([[8.44452950e-03, 4.70316318e-07, 9.69134460e-04, ...,
1.26985406e-05, 2.09367388e-02, 2.09367388e-02],
[8.40401908e-03, 4.70549780e-07, 1.32440905e-03, ...,
1.27048441e-05, 3.87611995e-03, 3.87611995e-03],
[8.35605789e-03, 4.70366332e-07, 1.08232592e-03, ...,
1.26998910e-05, 2.01078971e-02, 2.01078971e-02],
...,
[4.43502915e-03, 4.76372626e-07, 1.20787349e-03, ...,
1.28620609e-05, 2.29734955e-02, 2.29734955e-02],
[4.39246824e-03, 4.76665029e-07, 1.28495339e-03, ...,
1.28699558e-05, 2.10030563e-02, 2.10030563e-02],
[4.34670095e-03, 4.76611946e-07, 2.13343568e-04, ...,
1.28685226e-05, 2.21394456e-02, 2.21394456e-02]], shape=(94, 56))
learning_time = 0.008259499999894615
model.cluster_centers_ = array([[4.72494014e-03, 4.75969676e-07, 1.02195566e-03, ...,
1.28511813e-05, 2.23718714e-02, 2.23718714e-02],
[5.75070326e-03, 4.74875860e-07, 5.33510507e-05, ...,
1.28216482e-05, 1.10127753e-02, 1.10127753e-02],
[7.44249101e-03, 4.71939823e-07, 7.70875135e-04, ...,
1.27423752e-05, 1.42134605e-02, 1.42134605e-02],
...,
[7.96857621e-03, 4.71234548e-07, 1.29714275e-03, ...,
1.27233328e-05, 1.59741660e-02, 1.59741660e-02],
[6.65339052e-03, 4.73214119e-07, 3.72357747e-04, ...,
1.27767812e-05, 1.79516542e-02, 1.79516542e-02],
[7.27003925e-03, 4.72387215e-07, 6.89348874e-04, ...,
1.27544548e-05, 1.47742781e-02, 1.47742781e-02]], shape=(57, 56))
prediction_results = array([46, 2, 6, 7, 33, 13, 32, 23, 1, 24, 9, 25, 4, 5, 22, 29, 4,
22, 18, 12, 23, 5, 34, 5], dtype=int32)
prediction_time = 0.0005363000000215834
original_prediction_error = -0.008051099933083884
original_error_probability = -0.00483065995985033
original_correct_probability = 1.0048306599598504
single_features = [15.618214494343635, 7.6004666776394405, 6.7841264817384115, 1.7879450686357452, 17.1
24090556891094, 0.7088195777108689, 12.889373964544562, 11.908785959854038, 13.162692465691121, 6.89158
920434166, 5.33967075805177, 3.9319450010437262, 9.961876064331372, 13.108765915267208, 13.942708494465
094, 17.82004732026139, 2.296848545763412, 8.859022267364216, 8.101794108230877, 4.733586874925892, 12.
746290460290696, 3.692154988813258, 13.727621371095298, 7.328662392375239, 1.8841115021490815, 16.83011
220649213, 3.0917491765166583, 4.726566025768565, 15.906737735676474, 12.867646220024264, 17.7817925262
04956, 3.3615761684333716, 4.071637488287048, 0.9936986701899894, 15.035673662393187, 18.94784037157707
2, 11.694971155550427, 4.006220576536143, 14.384380269395765, 15.988050894607218, 8.711905406374807, 7.
308690486254071, 12.868008269897647, 15.489465626598916, 5.0804170625338765, 7.2315414270045615, 15.326
582799790788, 4.410793957507963, 12.728925682963894, 3.2274640777192984, 1.9246246240236011, 17.7731088
65699356, 12.304562529394946, 6.687535008706796, 2.6993677424959435, 13.671276052842952, 7.991106323396
537, 16.670902740813812, 5.58571762873137, 16.110158976953425, 18.669285343796545, 3.9688792758334275,
4.081196763103305, 8.515305262271145, 2.847535168333777, 18.423114875647954, 0.6810178909690904, 17.763
465692951755, 12.718124788601424, 11.65062932033963, 12.926041206006435, 9.542644934073104, 11.17921973
4295582, 15.511439671939012, 10.380595285957082, 1.2754988938279581, 11.471762577758255, 7.404774863926
761, 11.293912456629416, 14.454908051589614, 18.038565815355312, 12.212366578444048, 10.215852544740995
, 0.5893451757279498, 14.468888071124821, 6.763800187273147, 12.140860484136095, 10.008876109922342, 16
.58936508743847, 17.474826148726724, 12.213576629453831, 9.275324942277019, 11.009727502512552, 14.6022
95549844607]
single_center = 12
single_target = 0
prediction_results = array([54, 29, 48, 11, 26, 17, 41, 43, 28, 12, 47, 13, 14, 28, 7, 25, 32,
16, 3, 27, 1, 34, 33, 50, 11, 45, 49, 27, 15, 41, 25, 0, 13, 40,
10, 46, 9, 13, 38, 15, 56, 50, 41, 24, 8, 21, 52, 36, 1, 0, 11,
25, 20, 48, 55, 33, 3, 54, 30, 51, 18, 13, 13, 31, 22, 44, 17, 25,
1, 9, 41, 5, 39, 24, 35, 4, 53, 50, 39, 38, 6, 20, 35, 17, 38,
48, 20, 14, 2, 37, 20, 42, 23, 19], dtype=int32)
prediction_error = -0.1121116038713784
short_error_probability = -0.0011926766369295574
short_correct_probability = 1.0011926766369295
prediction_error_difference = 0.10406050393829452
error_probability_difference = 0.003637983322920772
correct_probability_difference = 0.003637983322920979
Error Correct
Original -0.004831 1.004831
Shortened -0.001193 1.001193

```

Рисунок 1.2 — Виконання

Метод еталонів вираховує належність даних до заданих класів на основі їх відстаней.

Для згенерованої вибірки помилки розпізнавання майже не відрізняються, але загалом для вибірки з однією ознакою похибка менше.

2 Вибірка Титаніку

Вибірка є даними про пасажирів човна Титанік, що потонув; має такі ознаки:

- PassengerId: ідентифікатор;
- Survived: 0 — не вижив, 1 — вижив;
- Pclass: клас, 1 або 2 або 3;
- Name: ім'я пасажирів;
- Sex: male або female;
- Age: вік;
- SibSp: кількість братів/сестер або дружини на борту;
- Parch: кількість батьків або дітей;
- Ticket: номер квитка;
- Fare: вартість квитка;
- Cabin: каюта;
- Embarked: порт, на якому сіли: С — Чербург, Q — Квінстаун, S — Саутхемптон;

Пропущені дані є у стовпці віку. Їх було замінено на середнє значення віку. Відкидати рядки не став, бо чим більше вибірка, тим краще.

Відкинуто ознаки PassengerID, Pclass, SibSp, Parch, Ticket, Fare, Cabin, Embarked. Залишено Survived (цільова ознака), Name, Sex, Age. Після виводу Name теж прибрано.

```
import os
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import KNNImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.svm import SVC

current_folder = os.path.dirname(os.path.abspath(__file__))
file_path = os.path.join(current_folder, "..", "train.csv")
df = pd.read_csv(file_path)

# Remove unnecessary columns
unnecessary =
"PassengerId,Pclass,SibSp,Parch,Ticket,Fare,Cabin,Embarked".split(",")
df = df.drop(unnecessary, axis=1)
print(df)

# Remove name
df = df.drop("Name", axis=1)
# Count missing values
print(df.isnull().sum())

# Convert gender to 0 and 1
le = LabelEncoder()
df["Sex"] = le.fit_transform(df["Sex"])

# Replace missing ages with average
imputer = KNNImputer(n_neighbors=7)
df["Age"] = imputer.fit_transform(df[["Age"]]).ravel()
print(df)

# Split dataset
target_feature = df.Survived.to_numpy()
survived_scores = pd.DataFrame(target_feature, columns=["Survived"])
df = df.drop(["Survived"], axis=1)
X_train, X_test, y_train, y_test = train_test_split(df, target_feature)

# Modeling results
kmeans_model = KMeans()
kmeans_model.fit(X_train)
kmeans_results = kmeans_model.predict(X_test)
print(f"{kmeans_model.cluster_centers_ =}")
print(f"{kmeans_results =}")
```

```

kneighbors_model = KNeighborsClassifier(n_neighbors=7)
kneighbors_model.fit(X_train, y_train)
kneighbors_results = kneighbors_model.predict(X_test)
print(f"{kneighbors_results = }")

forest_model = RandomForestClassifier()
forest_model.fit(X_train, y_train)
forest_results = forest_model.predict(X_test)
print(f"{forest_results = }")

logistic_model = LogisticRegression()
logistic_model.fit(X_train, y_train)
logistic_results = logistic_model.predict(X_test)
print(f"{logistic_results = }")

svc_model = SVC()
svc_model.fit(X_train, y_train)
svc_results = svc_model.predict(X_test)
print(f"{svc_results = }")

# Comparing models
testing_indeces = list(X_test.index)
corrects = [int(survived_scores.iloc[i].Survived) for i in
testing_indeces]

# Plotting graphs
models = ["KNeighbors", "RandomForest", "Logistic", "SVC"]
accuracies = [
    sum([int(abs(corrects[i] - model_result[i])) for i in
range(len(corrects))])
    for model_result in [
        kneighbors_results,
        forest_results,
        logistic_results,
        svc_results,
    ]
]

resulting_dictionary = dict()
for index in range(len(models)):
    m = models[index]
    a = accuracies[index] / len(testing_indeces)
    resulting_dictionary[m] = a
print("Errors in each model:", resulting_dictionary)
plt.title("Number of errors in each model")
plt.bar(models, accuracies, color="g")
plt.show()

```

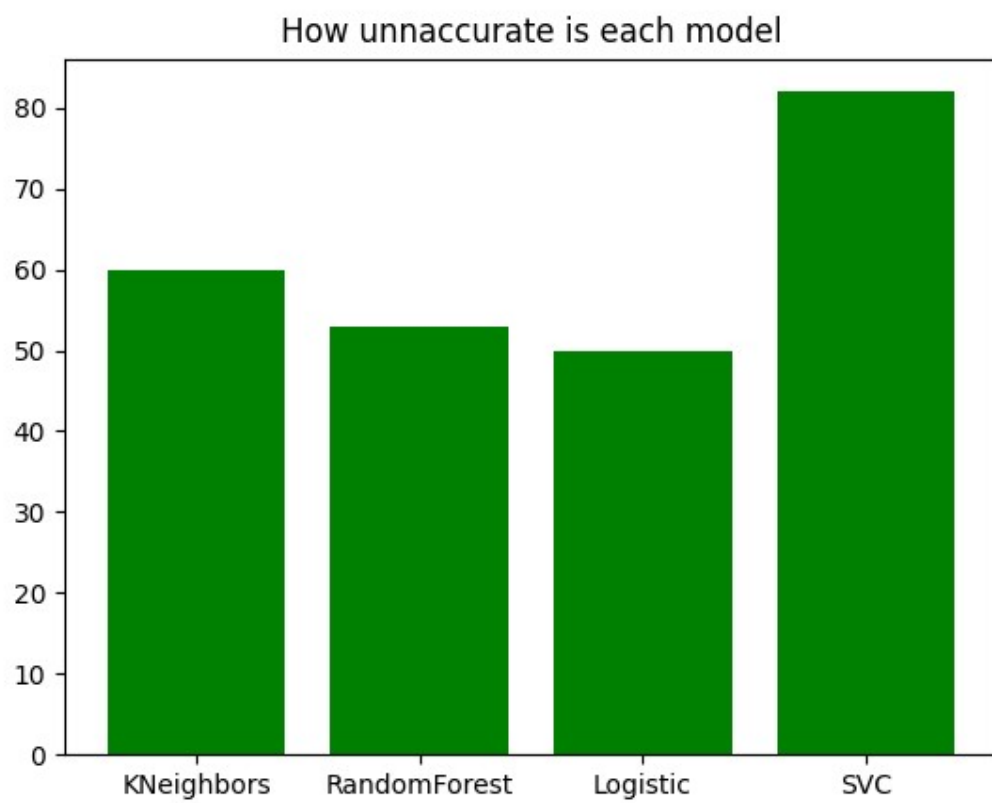


Рисунок 2.1 — Результати порівняння

	Survived	Name	Sex	Age
0	0	Braund, Mr. Owen Harris	male	22.0
1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0
2	1	Heikkinen, Miss. Laina	female	26.0
3	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0
4	0	Allen, Mr. William Henry	male	35.0
..	...	...	...	...
886	0	Montvila, Rev. Juozas	male	27.0
887	1	Graham, Miss. Margaret Edith	female	19.0
888	0	Johnston, Miss. Catherine Helen "Carrie"	female	NaN
889	1	Behr, Mr. Karl Howell	male	26.0
890	0	Dooley, Mr. Patrick	male	32.0

[891 rows x 4 columns]

```
Survived    0
Sex         0
Age        177
dtype: int64
```

	Survived	Sex	Age
0	0	1	22.000000
1	1	0	38.000000
2	1	0	26.000000
3	1	0	35.000000
4	0	1	35.000000
..	...	...	...
886	0	1	27.000000
887	1	0	19.000000
888	0	0	29.699118
889	1	1	26.000000
890	0	1	32.000000

[891 rows x 3 columns]

```
kmeans_model.cluster_centers_ = array([[ 0.67322835, 29.56173923],
    [ 0.71428571, 58.44285714],
    [ 0.44680851, 14.35106383],
    [ 0.63636364, 46.50757576],
    [ 0.65185185, 21.55185185],
    [ 0.58823529, 37.15882353],
    [ 0.46153846,  3.59205128],
    [ 1.         , 71.71428571]])
```

Рисунок 2.2 — Результати виконання, частина 1

```

kmeans_results = array([4, 6, 0, 0, 4, 5, 5, 3, 0, 4, 0, 3, 0, 4, 0, 0, 1, 0, 0, 4, 5, 5,
0, 5, 6, 6, 4, 0, 6, 6, 0, 0, 0, 5, 3, 3, 4, 0, 4, 0, 4, 0, 4, 6,
0, 0, 3, 0, 0, 0, 3, 3, 0, 5, 4, 2, 1, 5, 2, 6, 4, 0, 0, 4, 0, 5,
0, 0, 4, 0, 6, 4, 0, 4, 0, 0, 0, 4, 4, 0, 5, 0, 2, 4, 0, 4, 0, 5,
4, 5, 0, 0, 4, 2, 1, 3, 4, 5, 1, 0, 0, 0, 3, 0, 0, 3, 0, 0, 5, 0,
1, 2, 0, 5, 6, 0, 4, 0, 0, 0, 4, 3, 0, 2, 6, 5, 1, 0, 0, 0, 0, 3,
0, 0, 6, 0, 0, 6, 0, 4, 2, 4, 4, 1, 4, 4, 4, 5, 4, 0, 3, 0, 4, 0,
4, 5, 4, 4, 3, 6, 4, 4, 0, 3, 0, 3, 0, 0, 4, 0, 0, 0, 5, 4, 4, 4,
3, 2, 1, 3, 5, 5, 2, 0, 5, 3, 0, 3, 0, 4, 4, 0, 4, 2, 5, 0, 4, 4,
0, 4, 0, 4, 3, 0, 4, 0, 0, 3, 4, 7, 3, 5, 2, 4, 6, 0, 6, 4, 0, 3,
3, 0, 2], dtype=int32)
kneighbors_results = array([1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0,
0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0,
1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0,
0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1,
0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1,
0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1,
0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1,
0, 0, 0], dtype=int32)
forest_results = array([1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0,
0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0,
1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0,
0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1,
0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1,
0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1,
0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1,
0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0,
0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1,
0, 0, 0], dtype=int32)
logistic_results = array([1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0,
0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0,
1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0,
0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1,
0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1,
0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1,
0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0,
0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1,
0, 0, 0], dtype=int32)
svc_results = array([0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0,
0, 0, 0], dtype=int32)

```

Рисунок 2.3 — Результати виконання, частина 2

Errors in each model: {'KNeighbors': 0.26905829596412556, 'RandomForest': 0.23766816143497757, 'Logistic': 0.2242152466367713, 'SVC': 0.36771300448430494}

Рисунок 2.4 – Помилки моделей

Отримані помилки:

- k-сусідів — 27%;
- випадковий ліс — 23%;
- логістична регресія — 22%;
- SVC – 37%.

Покращити результати можна збільшивши тестувальну вибірку.

ПИТАННЯ

Задача розпізнавання образів.

Віднесення даних до якогось класу в залежності від ознак.

Основні поняття теорії розпізнавання образів.

Навчання — процес побудови моделі.

Ознака — характеристика набору даних.

Розпізнавання — отримання результатів моделі.

Розбиття вихідної вибірки на навчаючу та тестову.

Потрібне, щоб модель могла працювати з ще не баченими даними.
Перемішує дані і розділяє їх на два масиви різних розмірів.

Навчання з учителем.

Це коли моделі задаються правильні значення вихідної ознаки.

Методи метричної класифікації.

Модель створює кластери, шукає координати еталонів (середніх екземплярів) кластеру, категоризує вхідні екземпляри в залежності від вихідної ознаки до найближчого центру.

Призначення пакету «Sklearn».

Машинне навчання та розпізнавання образів.

Що таке генеральна сукупність, вибірка, екземпляр, ознака?

Генеральна сукупність — всі об'єкти якогось виду.

Вибірка — частина генеральної сукупності, яка представляє її точно.

Екземпляр — один з об'єктів вибірки.

Ознака — якась властивість екземпляру: колір.

Вимоги до навчаючих вибірок даних.

1. Репрезентативність;
2. Розмір хоча б 100 екземплярів;
3. Відсутність пропущених значень;
4. Текстові дані або прибрані, або переведені у мітки.

Що таке репрезентативна вибірка даних?

Така, що достовірно представляє дані.

Чи повинна навчальна вибірка бути репрезентативною?

Так.

Чи повинна тестова вибірка бути репрезентативною?

Краще щоб була.

Чи впливає обсяг навчаючої вибірки на швидкість навчання?

Так.

Чи впливає репрезентативність навчаючої вибірки на точність класифікації екземплярів тестової вибірки?

Так.

Чи впливає репрезентативність тестової вибірки на точність класифікації екземплярів тестової вибірки?

Так, впливає, але не так, як репрезентативність навчальної.

Чи впливає репрезентативність тестової вибірки на точність навчання персептрона за навчаючої вибіркою?

Ні, репрезентативність тестової вибірки впливає лише на результати розпізнавання.

Чи залежить якість навчання від якості та обсягу навчаючої вибірки?

Так, залежить. Якщо обсяг недостатній, помилка може бути більшою. Якщо якісь дані відсутні, моделі не зможуть опрацювати їх.